

EcoHome Energy Agent

An AI agent for smart-home energy optimisation

Built with LangChain & LangGraph

1. Project Overview

The **EcoHome Energy Agent** is an intelligent assistant that advises households — those with solar panels, electric vehicles, and smart devices — on **when to run their devices** to minimise electricity costs and maximise use of their own solar generation. Rather than giving generic tips, the agent makes **data-driven, time-specific recommendations** grounded in real electricity prices, weather and solar forecasts, the household’s own usage history, and a knowledge base of energy-saving best practices.

Given a question such as “*When should I charge my EV tomorrow to minimise cost and maximise solar power?*”, the agent gathers the relevant data using its tools, retrieves matching best-practice tips, calculates potential savings, and returns a recommendation with a specific time window, a cost analysis, a solar consideration, and a savings estimate.

Note: this is a portfolio / learning project running on representative sample data; recommendations illustrate the capability rather than reflecting a specific real household.

2. System Architecture

The agent is implemented as a **LangGraph ReAct loop**: the language model reasons about what information it needs, calls a tool to obtain it, reads the result, and repeats until it can produce an answer. The agent itself contains no hard-coded energy logic — all capability lives in seven tools, and the data lives in a SQLite database and a Chroma vector store. This separation means the agent’s behaviour is driven by its system prompt and the tool descriptions rather than by branching code.

Components

Component	Role	Implementation
Agent (agent.py)	Reasoning	LangGraph <code>create_react_agent</code> , GPT-4o-mini, temp 0
Tools (tools.py)	Capability	7 tools: weather, pricing, 3× database, RAG, savings
External APIs	Live data	Open-Meteo (weather/solar) & Octopus Agile (prices), with mock fallback
Database	History	SQLite: energy-usage and solar-generation tables
Knowledge base	RAG	Chroma vector store over 7 energy-saving documents

3. Implementation Highlights

Real external APIs with graceful fallback

The weather and pricing tools call genuine free APIs — **Open-Meteo** for weather and solar irradiance, and **Octopus Agile** for real UK half-hourly electricity prices — and adapt their responses to the agent’s expected format (for example, aggregating 48 half-hourly Agile prices into 24 hourly slots and converting pence to pounds). If an API is unreachable, each tool falls back to a realistic mock so the system always runs and remains reproducible offline.

EcoHome Energy Agent — Architecture

A LangGraph ReAct agent that reasons over live data + a RAG knowledge base to give energy recommendations

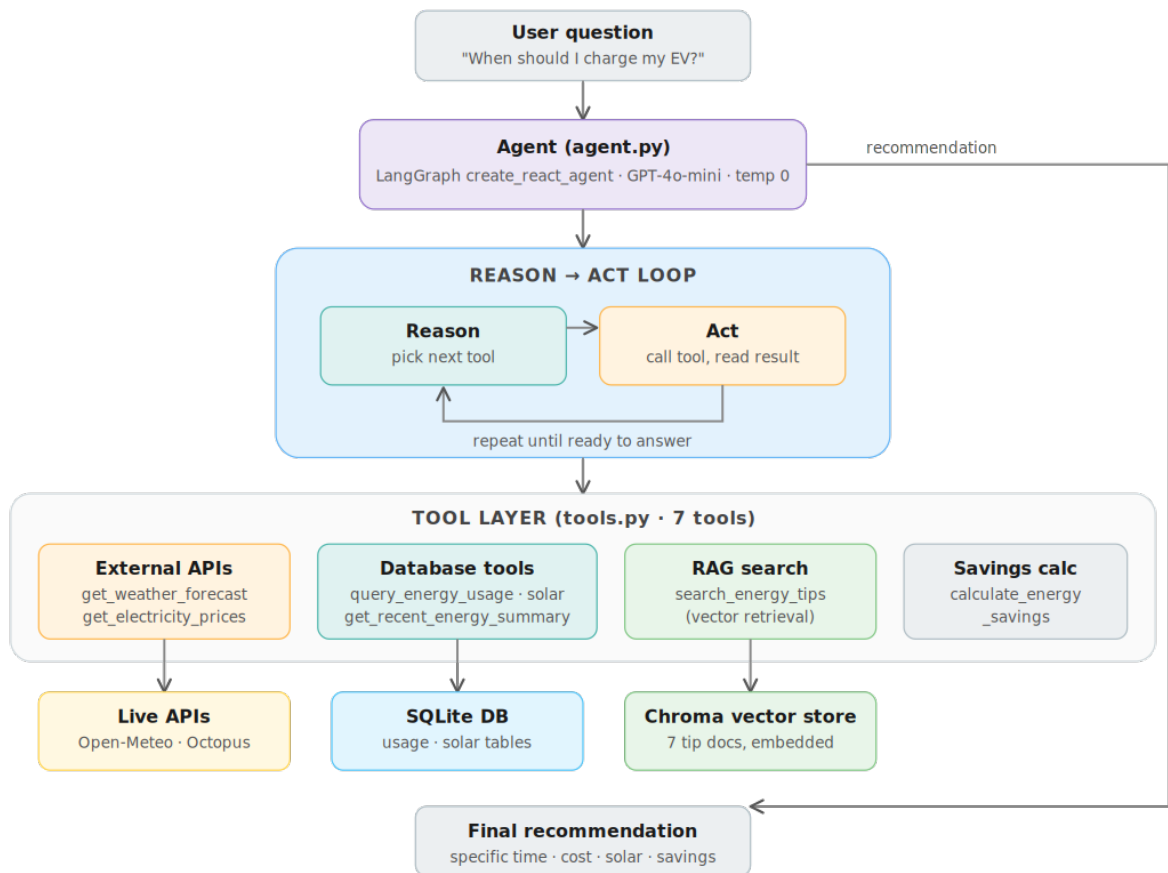


Figure 1: End-to-end architecture: question → agent → reason/act loop → tools → data sources → recommendation.

Retrieval-augmented generation (RAG)

Seven energy-saving documents (covering HVAC, smart-home automation, renewable integration, seasonal management, energy storage, device best practices, and general savings) are chunked, embedded with OpenAI embeddings, and stored in a Chroma vector database. The `search_energy_tips` tool retrieves the most semantically relevant chunks for a query, so the agent’s advice is backed by a curated knowledge base rather than the model’s memory alone.

System prompt as the agent’s ‘program’

Because the ReAct agent has no branching logic, its behaviour is governed by a carefully engineered system prompt that defines its role, a five-step reasoning process (understand → gather data → retrieve tips → calculate savings → synthesise), its tool catalogue, and strict requirements for every recommendation (a specific time, cost analysis, solar consideration, and savings estimate). The prompt firmly instructs the agent to gather data via tools before answering, which is essential for grounded responses.

4. Evaluation & Results

The agent was evaluated across **12 test cases** spanning EV charging, thermostat settings, appliance scheduling, solar maximisation, battery storage, historical analysis, and cost-savings calculations. Two evaluators were used: an **LLM-as-judge** scoring response quality (accuracy, relevance, completeness, usefulness) with a rule-based fallback, and a **rule-based tool-usage evaluator** scoring whether the right tools were selected (appropriateness) and all needed tools were used (completeness).

Overall results

Metric	Score	Meaning
Overall score	82.7%	Aggregate across all six metrics
Response quality	90.4%	Accuracy, relevance, completeness, usefulness
Tool usage	67.3%	Tool appropriateness and completeness
Tests passed	7 / 12	Both response and tool score ≥ 0.6

Per-metric averages

Metric	Avg	Assessment
Accuracy	0.89	Strong
Relevance	0.97	Strong
Completeness	0.86	Strong
Usefulness	0.90	Strong
Tool Appropriateness	0.69	Inconsistent (50% of tests below 0.6)
Tool Completeness	0.65	Inconsistent (58% of tests below 0.6)

Interpretation

Response quality is excellent (90.4%): the agent’s recommendations are accurate, relevant, complete, and actionable, as rated by the LLM judge. **Tool usage is the weaker axis** (67.3%), and the evaluation correctly diagnoses why: the agent often produces an excellent answer using

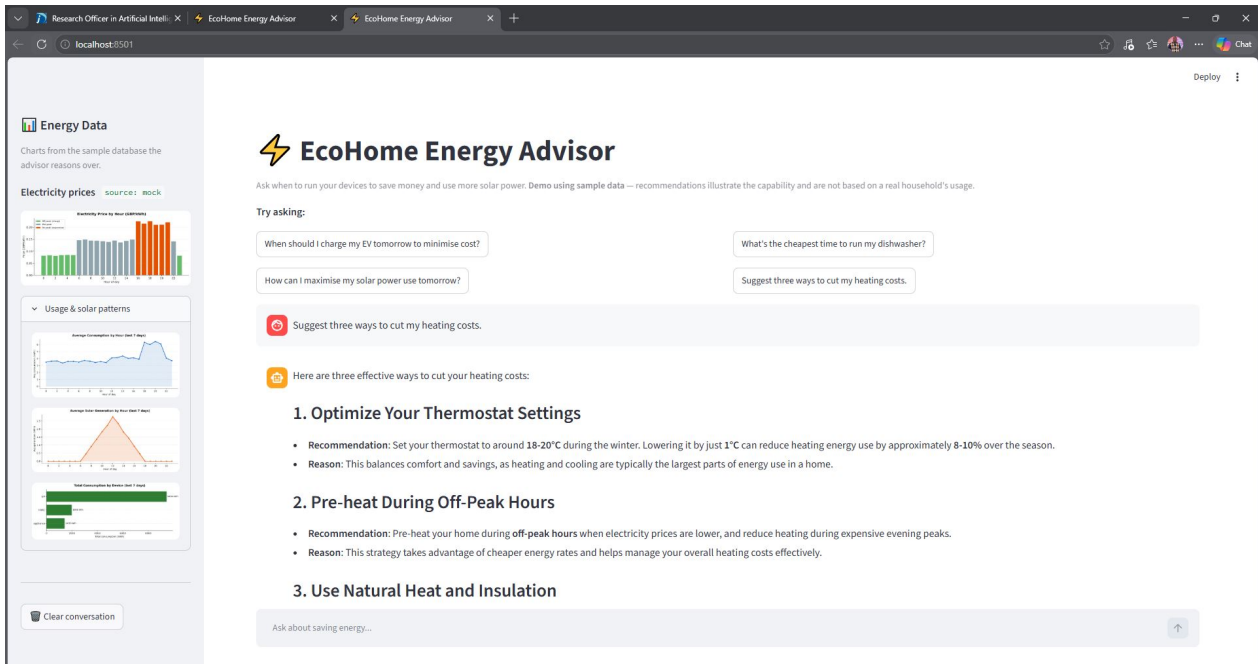


Figure 2: The demo app: the agent’s dishwasher recommendation with cost analysis, and the “Tools used” line confirming a real tool call. Live data charts appear in the sidebar.

fewer tools than the ideal set specified for a question. Notably, the report’s weakness detection flags this as *inconsistent* — the averages look acceptable but a large share of individual tests score low — a pattern a simple average would have hidden.

This is a credible, honest result: strong response quality with a specific, well-characterised area for improvement (encouraging the agent to gather data more thoroughly for multi-faceted questions). The evaluation system surfacing and explaining this — and generating targeted recommendations to address it — demonstrates that both the agent and its evaluation pipeline work as intended.

5. Interactive Demo Application

Beyond the notebooks, the same agent is wrapped in a **Streamlit web application** that provides a chat interface alongside live charts of the underlying energy data. Users can ask questions in natural language, see which tools the agent called for each answer (transparency into its reasoning), and explore the electricity-price curve, usage patterns, solar-generation profile, and device breakdown in the sidebar.

The example below shows the agent answering a dishwasher-scheduling question: it recommends a specific off-peak window (00:00–05:00), gives a full cost analysis with actual prices, and — via the “Tools used” line — shows that it called the real `get_electricity_prices` tool to ground its answer.

A second example shows the agent maximising solar use — reasoning over forecast solar irradiance (peaking at midday) to recommend running devices in the 11:00–14:00 window.

6. Summary

The EcoHome Energy Agent demonstrates an end-to-end agentic AI system: a LangGraph ReAct agent that integrates multiple tools (real external APIs, a historical database, and a RAG knowledge base) to deliver grounded, personalised energy recommendations. It is supported by a comprehensive evaluation framework and a working interactive demo.

Key technologies: LangChain · LangGraph · ChromaDB · OpenAI · SQLAlchemy / SQLite · Streamlit · matplotlib

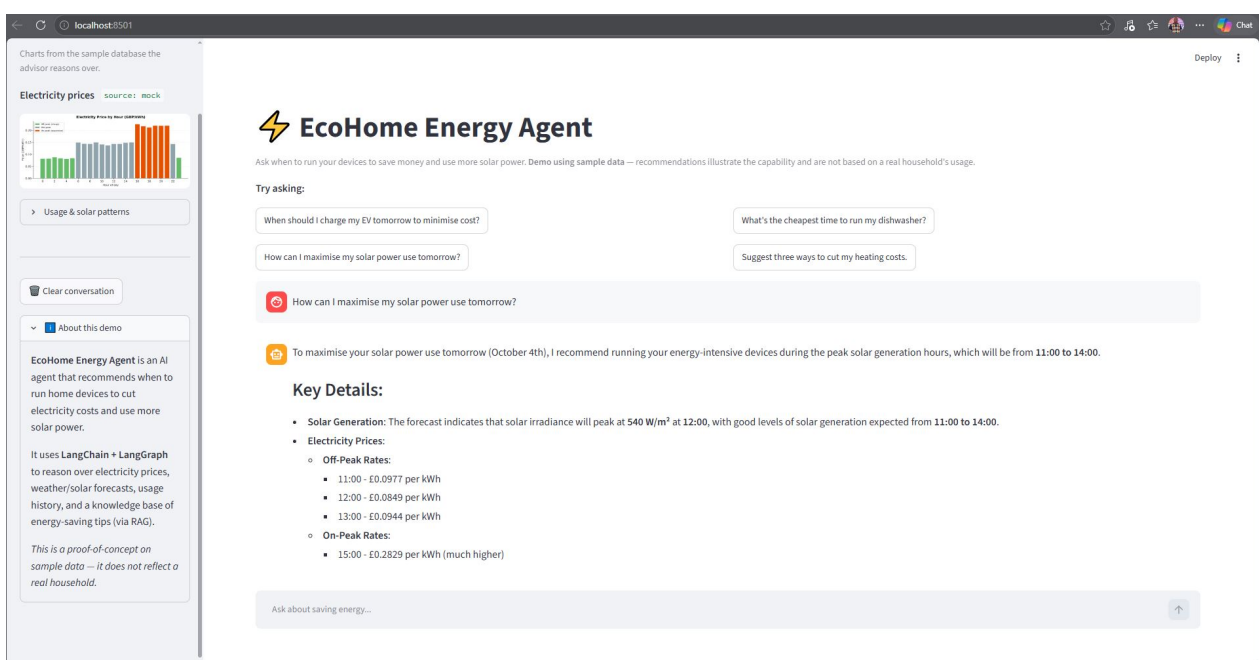


Figure 3: Solar-maximisation recommendation: the agent identifies the peak solar-generation window from the forecast.